

WEEK 14

治理与版本

数据像代码一样发布、回滚、追责——从“实验工具”到“能上生产、能过监管”的最后一公里

01 Branch

数据分支化

lakeFS · Git 模型

02 Bind

原子绑定

6+ 类一个 release_id

03 Impact

血缘 + 影响

OpenLineage 全链路

04 Compliance

合规白皮书

工程产物自动汇聚

05 Canary

灰度 + 回滚

4 阶段 · 秒级回滚

本周划重点

前 13 周,我们把 RAG、Agent、评测、可观测、GraphRAG 全建出来了。可有一个问题一直悬着——业务说“昨天回答没问题、今天突然不对”,你能在 5 秒内回到“昨天那套数据+索引+Prompt+模型”吗?这一周就补这最后一根主线:把 AI 系统的版本治理,做到软件工程级——能发布、能回滚、能追责、能过监管。

LESSON 01 · DATA AS CODE

把数据像代码一样管理：分支 / 提交 / 合并 / 标签

前 13 周做完了 RAG/Agent/评测/可观测/GraphRAG——还差最后一根工程主线没接好

真实场景 · "昨天没问题今天不对", 你能 5 秒回到昨天吗

我经手过一次真事故:一份产品手册被人覆盖式更新, 当晚 RAG 就开始答错, 客户投诉一片。团队想回滚——可数据是"覆盖式更新"的, 旧版本根本没留, 只能连夜从三天前的备份里捞、手工拼, 折腾到凌晨四点。你想想, 代码要是这么管——不开分支、直接改主干、出错没法 revert——早被开除了。可数据, 大部分团队至今还这么干。这一讲把数据也升级到 Git 时代:用 lakeFS 引入"分支"模式, 让数据变更像代码一样可 PR、可回滚、可灰度。

咱们这节聊 4 件事:

为什么 2026 年产线数据还在用"覆盖式更新 + 备份"是一笔工程债

数据治理三代演进——大部分团队卡在"快照"这一代, 差在哪

Git 模型怎么 1:1 搬到数据: Branch / Commit / Tag / Merge / Revert

lakeFS 和 Iceberg / Polaris 是"组合"不是"替代"——2026 的目录格局怎么选

DATA NEEDS GIT

数据治理的下一步不是"更精细的备份"——是把 Git 模型完整搬到数据上

代码能开分支开发、不影响主线、PR 合并、出错回滚——数据凭什么不行

传统数据治理这些年走了三步:数据备份 → 数据快照(Iceberg time travel)→ 数据资产化(Week 6)。但有一道根本的工程鸿沟一直没填——代码那套"开分支、隔离开发、PR 合并、一键回滚"的能力,数据一直没有。lakeFS 就是来填这道沟的:给 Lakehouse 引入 Git 模型——branch / commit / merge / tag / revert 全都有。

代码世界(早就有)

- git branch 开发
- PR review 合并
- git revert 回滚
- tag 锚定版本
- 出错零压力

数据世界(以前没有)

- 覆盖式更新
- 出错只能捞备份
- 一改影响所有人
- 版本靠文件名
- 出错就是事故

lakeFS 补齐(2024 起)

- 数据也能开分支
- 数据 PR + Merge
- 数据 revert 秒级
- tag = release 锚点
- 数据变更零压力

THREE GENERATIONS

数据版本治理的 3 代演进——你的团队在哪一代

大部分团队卡在 G2 快照，升到 G3 分支不需要换整个 Lakehouse

代际	核心能力	解决什么	局限
G1 · 备份	定期复制全量数据	出大事故能恢复	粒度粗 / 时延高 / 不能开发
G2 · 快照	Iceberg / Delta time travel	可读历史时刻数据	主分支唯一 / 一改影响所有人
G3 · 分支	lakeFS Git-like branches	并行开发 + 隔离 + 合并	需要工程化 PR 流程配合

关键判断

关键点:升到 G3 不用换掉你的 Iceberg,也不用换 S3。lakeFS 是"覆盖在 S3/HDFS 之上"的一层元数据,用 COW(copy-on-write)开分支几乎零成本、零复制。托管版 lakeFS Cloud 每月几百美金起、自托管开源免费。别被"又要上一套新系统"吓退——它是加一层,不是推倒重来。

GIT MODEL FOR DATA

Git 模型搬到数据——概念 1:1 对应，工程师上手零摩擦

你懂 Git，就懂 lakeFS；不用学新范式，只是换个对象

Branch · 分支

- main：生产数据
- staging：预上线
- experiment：实验
- 分支间互相隔离
- 改动不影响别人

Commit + Tag

- commit：一次数据变更
- tag：可寻址版本号
- release-2026-05-18
- release manifest 锚点
- 不可变 + 可回滚

Merge + Revert

- merge：把分支合并
- revert：撤销某次变更
- PR review 配套
- 保留完整历史
- 可审计 + 可追责

老司机说

这套映射最大的价值，是“零学习成本”。你团队里只要有人会用 Git，他就会用 lakeFS——同样的心智模型：开分支干活、提交、发 PR、审、合、错了 revert。工程师不用重新学一套黑话，这是它能落地的关键。

LAKEFS WORKFLOW

OmniSupport 里的 lakeFS 数据分支 workflow——数据 PR 长这样

从开分支改数据，到评测、影响分析、合并、秒回滚，一条命令链走完

```
# 1. 改"产品手册 V3.7"——开分支，不碰生产
$ lakectl branch create lakefs://omni/staging-docs-20260518 \
  --source lakefs://omni/main

# 2. 在分支上更新数据 (Dagster / Spark / Python)
$ python pipelines/update_docs.py \
  --target lakefs://omni/staging-docs-20260518/docs/

# 3. 提交，并把数据变更绑到代码 PR
$ lakectl commit lakefs://omni/staging-docs-20260518 \
  -m "Update handbook v3.7" --metadata git_pr=PR-142

# 4. 在分支上跑评测 + 影响分析 (不碰生产用户)
$ omni eval run --branch staging-docs-20260518 --baseline main
# faithfulness 0.91 vs 0.89 (+0.02) ✓ | 受影响 chunks 1,247

# 5. PR 合并 (两人审 + 自动回归)
$ lakectl merge staging-docs-20260518 main --reviewer alice@team

# 6. 出问题秒回滚
$ lakectl revert lakefs://omni/main --commit ed42a3...
```

老司机解读

branch create 立刻可用 · COW 开分支不复制数据,几乎零成本、零等待

metadata git_pr=PR-142 · 把数据变更和代码 PR 绑一起,这是本周治理的核心

eval run --branch · 在分支上跑评测,不影响生产用户,Week 11 评测在分支上做

merge 需 reviewer · 数据变更走"两人审",和代码 PR 同等严肃

revert 秒级 · 出错切回上一个 commit,用户感受不到中间过程——这就是"数据也能 Git"

LAKEFS ≠ ICEBERG

lakeFS vs Iceberg / Polaris——不是替代是组合，这点最容易误解

很多工程师以为"上 lakeFS 就要换掉 Iceberg"——错，它们各管一层

层级	解决什么	工具代表	在 OmniSupport 里
Storage	存原始字节	S3 / GCS / HDFS	存所有 Parquet / JSON / 文档
Table Format	表 schema / 快照 / 演进	Iceberg / Delta / Hudi	silver / gold 表用 Iceberg
Repo / Branch	跨表跨格式的分支隔离	lakeFS	main / staging / experiment
Catalog	统一目录 + REST + 多引擎	Polaris / Unity / Nessie	2026 新格局,下页细讲

关键判断

Iceberg 管"表级快照 + Schema 演进",lakeFS 管"整个湖的 git 层 + 跨表分支隔离",两者是不同层,生产里并存——Iceberg 表就存在 lakeFS 分支里。你要是把它们当成"二选一",就白白丢了一半能力。这是本周第一个要纠正的直觉。

INDUSTRY 2026

Data as Code 已是工程基线——2026 的目录格局刚洗过牌

所有主流 Lakehouse 都内嵌“数据分支”,这不再是先进做法, 是准入门槛

方案	定位	2026 现状 / 建议
lakeFS	Git for Data (湖层分支)	开源、S3 兼容,数据 PR/Merge/Revert 首选
Apache Polaris	Iceberg REST 目录	2026-02 升为 Apache 顶级项目,多引擎互通
Unity Catalog (OSS)	治理 + 血缘 + 目录	Databricks 开源捐 Linux 基金会,上 Databricks 首选
Project Nessie	Iceberg 原生分支	全 Iceberg 团队的 git-like 分支引擎
Snowflake Git 集成	仓库内建 Git	全 Snowflake 团队用

老司机说

2026 一个大变化:开放目录标准打通了。Apache Polaris 今年 2 月升为顶级项目、Unity Catalog 也开源了,两者都实现 Iceberg REST API,Flink/Spark/Trino/StarRocks 一份目录到处用。我的建议:湖层分支用 lakeFS,目录用 Polaris 或 Unity(看你在不在 Databricks 生态),别再自己造目录胶水——这层 2026 已经卷完了。

WHEN TO ADOPT

lakeFS 上不上的 5 个判断标准——为痛点而上，不为先进而上

这张表是我给团队定的"准入清单"，符合 3 项以上再上

判断项	具体问题	阈值	不达标就别上
团队规模	数据/AI 工程师数量	> 5	太少没必要工程化
变更频率	数据更新 PR / 周	> 5	低频用 commit/script 够
事故痛点	"回滚困难/一改全断"事故	近半年 > 3 次	不痛就不用上
工程能力	Git PR 文化 / CI 经验	齐全	没基础 lakeFS 也用不起来
监管压力	需提供"数据版本审计"	是	是 = 必须,否 = 加分

关键判断

用法:符合 3 项以上 → 上 lakeFS;只符合 2 项 → 先用 Iceberg time travel 撑半年再看;不到 2 项 → 别折腾。我见过太多团队,痛点还没到就急着上一堆治理工具,结果工具比业务还重,这叫过度工程,老板一算 ROI 就砍了。

LESSON 02 · ATOMIC BINDING

五元原子绑定：把数据 / 索引 / Prompt / Skill / 图 锁进一个 release_id

Week 8 讲过 release manifest, Week 9-13 每周加一类对象——这一讲收口

真实场景 · "昨天那条回答是哪个版本生成的"

有次线上一条回答出了问题,合规部门追过来问:"这条回答,当时用的是哪份数据、哪个 Prompt、哪个模型?" 负责的工程师当场卡壳——数据是运维改的、Prompt 是他自己前天调的、模型上周升过、索引又是另一个人重建的,四五个东西各自有各自的版本,谁也说不清那一刻它们是什么组合。查了整整两天才拼出现场。这一讲我把它收口:AI 系统的一次发布,必须把 6 类对象锁进一个 release_id——客户报任何一个 bad case,你只要拿这一个 ID,整个"现场"瞬间完整恢复。这才叫 production-grade。

咱们这节聊 4 件事:

- 为什么"分别版本数据/索引/Prompt/模型"在生产里几乎一定踩坑
- release manifest 的 6 大对象 + 9 大段落——每段对应前面某一周
- release_id 命名规范 + 自动生成器——为什么绝不能手写
- 2026 供应链可信:model/system cards + 快照钉死 + Sigstore 签名溯源

ONE RELEASE = SIX OBJECTS

AI 的"一次发布" = 6 类对象的一次性原子绑定——任何分开都是工程债

同样的代码 + 不同的数据/Prompt/模型, 行为可以完全不同——这是 AI 和传统软件的根本差异

传统软件 release = git tag = 一个 commit hash, 代码定了行为就定了。AI 系统不一样: 同一份代码, 换一份数据、换一个 Prompt 版本、换一个模型快照, 行为可以天差地别。所以 AI 的"一次发布", 必须把这 6 类对象一起锁死: data + index + prompt + model + skill + graph = ONE release_id。

DATA · 数据+索引

- lakeFS commit / tag
- pgvector / ES snapshot
- "用了哪份数据"
- 影响: 召回内容
- Week 6/14 L1

INFERENCE · 推理

- prompt v3.2.1 (Jinja)
- model 快照钉死
- "用了哪个推理"
- 影响: 生成行为
- Week 8

AGENT · 智能体

- skill_pack v1.4
- graph schema v1.2
- "用了哪些工艺/关系"
- 影响: 执行 + 归纳
- Week 9/13

UNIFIED RELEASE MANIFEST

OmniSupport 的统一 release manifest——9 大段把前 13 周一网打尽

release_id 是这一切的"主键", 拿这一个 ID 整个现场完整恢复

```
# release/manifests/rag-v2026.05.18-001.yaml
apiVersion: omnisupport.rag/v5
metadata:
  release_id: rag-v2026.05.18-001
  released_by: alice@dataengineer
  approved_by: [bob@platform-lead, carol@compliance] # 三方审
spec:
  data: # 1. 数据 (Week 6/7/14)
    lakefs_tag: data-2026-05-18-v3.7.2
    iceberg_snapshots: {tickets: snap_..., docs: snap_...}
  index: # 2. 索引 (Week 7/8)
    pgvector_snapshot: idx-pg-20260518-093200
    chunk_strategy_version: chunk-v1.4
  prompt: {rag_answer: v3.2.1, intent_router: v1.0.5} # 3. Prompt
  model: # 4. 模型 (快照钉死)
    generator: openai/gpt-4o-2024-08-06 # 或 anthropic/claude-opus-4-8
    reranker: BAAI/bge-reranker-v2-m3@2024-06
  skill_pack: {version: 1.4.0, digest: sha256:abc...} # 5. Skill
  graph_release: {schema_version: 1.2.0, entity_count: 18420} # 6. 图
  eval: {eval_set: rag_qa_golden_v2.3.1, faithfulness: 0.892} # 7. 评测
  business_slo: {first_resolution_rate: 0.68, csat: 4.2} # 8. SLO
  rollout: # 9. 灰度回滚
    auto_rollback_on:
      - {metric: pii_leak_rate, threshold: 0}
      - {metric: faithfulness, threshold: "<0.86"}
```

老司机解读

9 大段 = Week 1-13 所有工程对象一网打尽 · 每段对应一个 PPT 周次

release_id 是"主键" · 拿这一个 ID, 整个现场完整恢复

approved_by 多人签 · 上线必须"代码+数据+合规"三方审, 不能单人决策

model 快照钉死 · 别写 gpt-4o 这种滚动别名, 要写到日期版本, 否则供应商悄悄换权重你都不知道

auto_rollback_on · 退化条件显式声明, 含 PII(合规红线)+ faithfulness(质量底线)

SUPPLY CHAIN · 2026

2026 治理新维度：供应链可信——不只"绑了什么"，还要"证明没被动过"

模型/数据也是"依赖"，也需要 cards + 签名 + 溯源，跟软件供应链一个道理

能力	做什么	2026 工具 / 标准
Model / System Card	模型能力、限制、风险、评测随 release 附上	Anthropic/OpenAI system card,已成发布标配
快照钉死 (pin)	模型写到日期版本,禁用滚动别名	claude-opus-4-8 / gpt-4o-2024-08-06
制品签名	给 manifest / 索引 / 模型加密签名防篡改	Sigstore / cosign,ed25519
供应链清单	AI 版 SBOM:列全数据源、模型、依赖	SLSA + AI-BOM,监管开始要

老司机解读

这是 2026 才真正热起来的一块。以前我们只关心"绑了哪些版本",现在监管和安全还要你证明"这些版本没被人偷偷改过"。道理跟软件供应链一模一样——你 npm 装个包都要查签名了,凭什么模型、数据这种更要命的依赖反而裸奔?

落地判断:manifest 生成后用 cosign 签一下、模型一律 pin 到日期快照、每个 release 附上 model card——这三件小事,2026 做合规审计时能帮你省掉一大堆扯皮。别等监管问了才补。

RELEASE NAMING

release_id 命名规范——看似小事，决定 6 个月后你能不能查得动

命名搞乱了，回头查半年前某次回答用的什么版本，就是一场噩梦

策略	示例	优点	缺点
纯时间戳	2026-05-18T15-00-00Z	简单 + 唯一	人读不出"是什么"
SemVer	v3.2.1	人友好	不可单调递增 / 易冲突
Hybrid（推荐）	rag-v2026.05.18-001	可读 + 可解析 + 单调	需要工程约定
UUID	rel-a3f4b2c1d5e6	完全独立	不可读 / 不可排序

关键判断

直接抄这个:Hybrid = "服务名-v日期-序号" = rag-v2026.05.18-001。既人友好(一眼看出是哪天哪个服务的第几个版本),又机器友好(可解析、可单调排序)。别用纯 UUID——半年后出事故,你面对一屏 rel-a3f4b2c1 根本不知道哪个在前哪个在后。

RELEASE GENERATOR

Release Manifest 自动生成器——绝不要手写

手写必漏、必错版本；让工具自动收集 6 类对象、自动跑评测、自动打 tag

```
# release/generator.py
def generate_release_manifest(service="rag", released_by=None,
                             approved_by=None) -> dict:
    today = datetime.utcnow().strftime("%Y.%m.%d")
    seq = get_next_seq(service, today) # 当天第几个,自动递增
    release_id = f"{service}-v{today}-{seq:03d}"
    manifest = {
        "metadata": {"release_id": release_id, "released_by": released_by,
                     "approved_by": approved_by},
        "spec": {
            "data": collect_lakefs_state(), # 自动收集
            "index": collect_index_state(),
            "prompt": collect_prompt_versions(),
            "model": collect_model_versions(),
            "skill_pack": collect_skill_pack_state(),
            "graph_release": collect_graph_state(),
            "eval": run_eval_and_capture(), # 发布前自动跑评测
            "business_slo": collect_current_slo(),
            "rollout": load_default_rollout_policy(),
        }
    }
    payload = yaml.dump(manifest["spec"], sort_keys=True)
    manifest["metadata"]["digest"] = sha256(payload.encode()).hexdigest()
    return manifest # 任何字段变化 → digest 变 → 不可篡改证明

# $ omni release create --service rag --approver bob --approver carol
# Generated: rag-v2026.05.18-001 | Git tag (annotated) 已打
```

老司机解读

release_id 自动生成 · 单调递增 + 唯一,永远不会"同一天两个版本"撞名

collect_* 自动收集 · 不靠人工填,工程师只需一条"发布命令"

run_eval_and_capture · 发布前自动跑评测,结果 inline 写进 manifest——Week 11 评测强制接入

digest 整体 hash · 任何字段一动 hash 就变,这就是"不可篡改证明"

git tag annotated · 让 manifest 在 Git 里也可寻址,配合上页 cosign 签名,溯源闭环

HOW MANIFEST IS CONSUMED

Release Manifest 的 5 类消费方——做出来没人用 = 等于没做

manifest 不能只是一个躺在仓库里的文件，必须被这 5 类下游实际消费

消费方	具体怎么用	Week 关联
Runtime Service	启动时加载 manifest, 设 release_id 环境变量	Week 8
Observability	span 自动带 release_id, Phoenix/Jaeger 可查	Week 12
Evaluation	PR 触发评测时锁定 manifest 版本	Week 11
Audit	合规审计直接导出 manifest + trace	Week 14 L4
Rollback	omni release rollback <id> 一键切换	Week 8 L5

关键判断

这 5 类消费方在生产里同时用, release_id 才真正成为"AI 系统的核心 ID"。少接一类, manifest 就废一半: 不接 runtime, 它就管不住线上; 不接 observability, 你出事故还是对不上号; 不接 audit, 合规来查你还是手忙脚乱。

INDUSTRY 2026

Release Manifest 模式 = AI 工程化共识——各家都在收敛

不管叫 manifest、artifact 还是 lockfile，核心都是“多类对象原子绑定 + 签名溯源”

来源	做法	给你的启示
Anthropic / OpenAI	生产 pin 具体 model snapshot + system card	原子绑定的概念基础,快照必须钉死
MLflow / W&B Artifacts	模型+数据+评测绑成 immutable artifact	传统 ML 的"原子绑定"先驱,直接复用
Sigstore / SLSA	制品签名 + 供应链等级	2026 给 manifest/模型加签名,防篡改
Spotify Backstage	service/data 配置作可审计 YAML	开源服务治理事实标准,同构

老司机说

你不用纠结"自研 manifest 还是用现成的"——概念都在收敛:一次发布 = 多类对象一次锁定 + 可签名 + 可回滚。传统 ML 时代 MLflow/W&B 早就把"模型+数据+评测"绑一起了,你要做的只是把 AI 时代新增的 Prompt、Skill、图、合规红线也塞进同一个包。站在这些成熟实践的肩膀上,别从零发明,这是老司机和新手最大的区别。

LESSON 03 · LINEAGE & IMPACT

全链路血缘 + 影响分析：改字段之前先看"会炸到哪 47 个服务"

Week 6/12 学过血缘，这一讲升级到"AI 全链路影响分析"

真实场景 · 上游一句"下周改个枚举值"，下游血流成河

上游 PM 周一开会："下周准备给 `ticket.status` 加个枚举值。"听着人畜无害对吧？没血缘的团队怎么办：拉个会评估三小时、挨个问五个组、结果上线后还是漏了某个下游 `dashboard`，当天报表全空。有血缘的团队怎么办：工具 30 秒渲染出影响图——9 个 RAG 服务 + 23 个 `dashboard` + 5 个 Agent Skill 受影响，其中 8 个是 P0/P1，直接 @ 到对应负责人。这一讲我把血缘，从"事后查"，教你升级成"改之前秒看下游影响"。

咱们这节聊 4 件事：

为什么"上游开会通知下游"这套在 AI 时代根本行不通

血缘的 3 个查询方向(向上/向下/横向)各回答什么工程问题

AI 时代血缘的 5 个层次——从原始文档一路打通到 Agent 动作

2026 血缘现状:OpenLineage 进 CNCF、列级血缘成标配、把 Agent 动作也纳进来

PREVENT, NOT INSPECT

血缘的最大价值不是"事后查"——是"改东西之前秒看下游影响"

90% 的 AI 重大事故都是同一个公式: 上游改了 + 下游没人知道 = 事故

"上游改了一个东西" + "下游没人知道" = 事故。这个公式,你没法用"上游开会通知下游"来破——因为上游根本不可能知道下游有谁在用、用了哪个字段。真正的解法只有一个:建立机器可读的全链路血缘 + 自动影响分析。上游改任何东西之前,工具自动生成"影响报告",把受影响的下流 service/dashboard/Agent/Skill 全列出来。

Upstream · 向上

- "这答案是哪来的"
- 从 Agent/Service → 数据源
- 解决:故障复盘/合规审计
- OpenLineage 反向遍历
- 产出:data→answer 全链

Downstream · 向下

- "改了它影响谁"
- 从字段 → 所有下游消费
- 解决:变更影响分析
- OpenLineage 正向遍历
- 产出:受影响下游清单

Lateral · 横向

- "还有谁用这字段"
- 一个字段的同代消费者
- 解决:协调改造/共享
- DataHub 共用查询
- 产出:同代消费者清单

FIVE LINEAGE LAYERS

AI 时代血缘的 5 个层次——全打通才叫"全链路"

传统血缘只到"数据"层，AI 时代还要往下打通到 Service 和 Agent 动作

层级	记录什么	工具	关键 ID
Source	原始文档 / API / 数据库表	OpenLineage	doc_id / table_id
Data	数据资产 (Iceberg / Lakehouse)	OpenLineage + DataHub	asset_id (列级)
Index	向量库 / ES / 图	Custom span	index_release_id
Service	RAG / Tool / Skill	OTel + Week 12	service_name
Action	Agent 决策动作	Custom + OpenLineage	trace_id

关键判断

5 层用同一个图谱平台(DataHub / OpenMetadata)串起来,改任何一层都能秒级反查"上下文 + 影响"。2026 特别强调 Data 层要做到"列级血缘"——不是"这张表影响那张表",而是"tickets.status 这一个字段影响哪些下游",粒度差一级,影响分析准确度差十倍。

IMPACT ANALYSIS · CODE

影响分析的自动化实现——改字段前必跑，接进数据 PR

一条命令,递归 5 层下游,按类型聚合,标出 P0/P1 高危

```
# tools/impact_analysis.py
def analyze_impact(asset: str, change_type: str) -> dict:
    client = DataHubClient()
    # 1. 递归找所有下游消费者 (跨 5 层)
    downstream = client.get_downstream(
        asset=asset, depth=5,
        include_types=["dataset", "dashboard", "rag_service",
                       "agent_skill", "ml_model", "dbt_model"])
    # 2. 按变更类型算风险等级
    risk_items = []
    for node in downstream:
        if change_type == "breaking": sev = "P0" if node["criticality"]=="P0" else "P1"
        elif change_type == "rename":  sev = "P1" if "field_path" in node["dependency"] else "P2"
        else:                          sev = "P3" # additive
        risk_items.append({"downstream": node["name"], "type": node["type"],
                          "owner": node["owner"], "severity": sev})
    # 3. 出报告
    return {"asset": asset, "total_downstream": len(downstream),
           "high_risk": [r for r in risk_items if r["severity"] in ["P0", "P1"]],
           "recommended_action": "coordinate" if change_type=="breaking" else "notify"}

# $ omni impact analyze --asset tickets.status --change-type breaking
# total_downstream: 47 | rag_service:9 dashboard:23 agent_skill:5
# high_risk: 8 (P0:2, P1:6) | action: coordinate
```

老司机解读

get_downstream(depth=5) · DataHub 跨多层血缘自动遍历,不用工程师手查

change_type 分级 · breaking/rename/additive 对应不同风险,Week 8 契约演进学过

high_risk 列 P0/P1 · 这些必须协调改造再上线,不是通知一下就行

CI 集成 · 数据 PR 一提交自动跑,Reviewer 看着"影响范围"决定批不批

这套让"上游变更"从"开会问一圈"变成"看一份报告"——决策速度提升 10 倍



OPENLINEAGE EXTENDED

把 Agent 动作也纳入血缘——扩展 OpenLineage facet

传统血缘只追“数据流”，AI 时代必须追到“Agent 改了哪些数据”

```
# pipelines/lineage/extended.py
class AgentActionFacet(BaseFacet):
    """扩展 OpenLineage,支持 Agent 动作"""
    def __init__(self, action_type, tool_name, release_id, approver=None):
        self.action_type = action_type # rag / tool / skill
        self.tool_name = tool_name
        self.release_id = release_id

@with_lineage # 装饰器自动注入,业务代码不动
def ticket_update(payload: dict) -> dict:
    result = perform_update(payload)
    ol_client.emit(RunEvent(
        eventType=RunState.COMPLETE,
        run={"runId": payload["trace_id"], "facets": {
            "agent_action": AgentActionFacet(
                action_type="tool_call", tool_name="ticket_update",
                release_id=os.environ["RELEASE_ID"]
            )
        },
        inputs=[Dataset(name=c) for c in payload["evidence_chunk_ids"]],
        outputs=[Dataset(name=f"tickets.{payload['ticket_id']}")])
    return result
# 一次完整血缘: 用户 query → chunks → tool_call → ticket 变更
```

老司机解读

AgentActionFacet · OpenLineage 的“扩展点”,在标准协议之上加 AI 专用字段

inputs = chunk_ids · 把 Agent 决策的“依据”纳入血缘,上游一改就知道哪些决策受影响

outputs · Agent 改了哪些数据,下游报表/视图能反向追溯

@with_lineage 装饰器 · 自动注入,业务代码不动,复用 Week 12 的 instrumentor 模式

2026 意义 · Agent 会“动手改数据”,不把动作纳入血缘,出了事你连“谁改的”都查不到

LINEAGE TOOLS · 2026

血缘工具生态——2026 选型，OpenLineage 已进 CNCF

协议 + 平台双层：OpenLineage 是协议，DataHub / OpenMetadata 是图谱平台

工具	层级	特点	适用
OpenLineage	协议 (CNCF)	事件级、跨工具通用、列级血缘	所有 AI 项目都该用
Marquez	基础后端	OpenLineage 参考实现	小团队 / 自托管入门
DataHub	图谱平台	LinkedIn 出品 + 影响分析 UI	中大型企业
OpenMetadata	图谱平台	更现代 + 强 UI	中型团队 / 自托管首选
Monte Carlo / Bigeye	数据可观测	血缘 + 异常告警 + 影响范围	大企业 / 要产品级

老司机说

2026 推荐:OpenLineage 协议(它已经进了 CNCF、Airflow/dbt/Spark/Flink 全原生支持)+ OpenMetadata 平台——开源、跨工具、可控,是中小团队最优组合,而且开源零 license 费——商业数据可观测(Monte Carlo 一类)年费常在 \$50,000 以上,预算紧就用开源栈。选型别纠结:协议跟 OpenLineage、平台看规模挑一个图谱库,千万别手画 Confluence 血缘图——那玩意儿画完第二天就过期了。

IMPACT-AWARE PR

影响分析驱动的 PR workflow——数据 PR 必跑，高危必须下游 owner 批

类比：代码 PR 必跑 unit test，数据 PR 必跑影响分析



关键判断

这条 workflow 把"上游单方面改"变成"按影响范围协调"——这是数据治理从混乱到有序最关键的一步。规则很硬：影响范围里只要出现 P0/P1 下游，这个 PR 就必须由那个下游的 owner 点头才能合并。就像代码里改了公共接口必须让调用方 review 一样，数据世界终于也有了这套规矩。

COMMON PITFALLS

血缘 + 影响分析的 5 个反模式——"手动维护"和"不接 PR"最致命

90% 的"血缘项目失败", 都是死在这两个坑上

反模式	具体表现	后果	正确做法
手动维护血缘	画 Confluence 图	永远过期	OpenLineage 自动生成
只到表级血缘	不到字段级	改字段不知影响谁	列级血缘 + DataHub
影响分析不接 PR	工具有但人忘了用	事故照旧	CI 强制运行 + 报告必读
Agent 动作不入血缘	只追数据流	Agent 改的数据查不到	扩展 OpenLineage facet
血缘没 Owner	看到下游不知找谁	P0 故障无人响应	每个 asset 必含 owner

踩坑提醒

我见过太多"血缘项目"轰轰烈烈启动、半年后无疾而终,复盘下来几乎都是同两个原因:一是手动维护(画个图挂 Confluence,三个月没人更新就成了废纸),二是有工具但不接 PR(人总会忘记去查)。记住:血缘必须"自动生成 + 强制卡在 PR 上",靠人的自觉,这事一定黄。

LESSON 04 · COMPLIANCE WHITE PAPER

合规审计白皮书：让"AI 系统上线"满足监管的硬性要求

2026 年 AI 系统上线最大的卡点，往往不是技术、不是预算——是合规

真实场景 · 技术都跑通了，卡在合规三个月上不了线

我见过一个系统,模型、评测、可观测全做到位了,demo 惊艳,业务方拍板要上。结果卡在哪?合规。法务要一份"可解释 + 可追溯 + 可审计"的材料,团队没有,只能找律师、找合规顾问、照模板人工写——外部费用轻松 \$20,000 到 \$50,000、来回三周,拖了三个多月,黄花菜都凉了。这一讲我要告诉你:2026 年,合规白皮书根本不该人工写——Week 11-14 你已经产出的那些工程材料(manifest + trace + 血缘 + 评测),自动汇聚一下,就是白皮书。顺带,我还要纠正几个 2026 已经变天、但大部分 PPT 还在照抄的合规事实。

咱们这节聊 4 件事：

2026 合规实况(重要):美国 EO 14110 已废除、EU AI Act 精确时间线、中国 AI 标识法

为什么合规白皮书应该"工程产物自动汇聚",而不是人工写 3 周

白皮书的 8 段标准结构 + 自动生成器——每段对应前面某一周的产物

4 类合规红线 + 监管来查时的"30 秒响应" workflow

COMPLIANCE IN 2026

2026 合规变天了——别再照两年前的 PPT 讲监管

这一页是我特意加的：三个已经变化、但很多材料还在照抄旧版的事实

地区	2024 旧认知（很多 PPT 还在讲）	2026 真实情况
美国	拜登 EO 14110 是硬要求	EO 14110 已于 2025-01-20 被废除,联邦转去监管——别再引它
欧盟	EU AI Act 2025 全面生效	GPAI 执法 2026-08 起;高风险经 Digital Omnibus 推迟到 2027-12;AI 标识 2026-08
中国	生成式 AI 备案 + 安全评估	再加 AI 生成内容标识办法(GB 45438-2025,2025-09-01 起强制)

老司机解读

我特意把这页放最前面,因为"合规讲错"比"不讲"更糟——你拿着一个已经被废除的行政令去跟法务对齐,直接翻车。2026 的真实格局:美国联邦层面松了(约束主要来自加州等州法),欧盟仍是最严但高风险义务被推迟给了喘息期,中国这边最实在——C 端服务不光要备案,生成的内容还必须打"AI生成"标识,微信、抖音、小红书全在管。

给国内团队的判断:你今天上一个 C 端 AI 服务,备案 + 内容标识是硬门槛,这俩没有,平台直接不让你上。

COMPLIANCE FRAMEWORKS

主流合规框架——你的系统要满足哪一个

跨国业务：按最严的准备（EU）+ 当地最直接的落地（本地监管）

框架	适用范围	硬性要求	2026 状态
EU AI Act	欧盟市场	透明度/可解释/可追溯/监督	最严;高风险推迟到 2027-12
中国备案 + 标识	中国大陆 C 端	备案 + 安全评估 + 内容标识	GB 45438 已强制,硬门槛
NIST AI RMF	美国 + 国际标杆	风险管理 + GenAI Profile	自愿,政府采购参考
ISO/IEC 42001	国际通用	AI 管理体系认证	自愿认证,投标加分
行业监管	金融/医疗/法律	行业特定 SLA + 审计	各行业自己定,通常最细

关键判断

给跨国业务一个省心策略:按 EU AI Act(最严)搭工程底座,再叠加当地监管(最直接)的具体要求。因为最严那套的"可追溯/可审计/可解释"你一旦做到了,其他框架基本是它的子集,提前为最严做,反而最省钱。

WHITE PAPER STRUCTURE

合规白皮书的 3 大组成 + 8 个段落——全部来自已有工程产物

不是“为合规重新做工作”，是“把已有产物按合规视角重新组织”

OVERVIEW · 概览

- 1. 系统描述(业务/范围)
- 2. 数据来源与契约
- 3. 模型与算法
- 让审计员懂“做了什么”
- 来源:项目文档 + 契约

EVIDENCE · 证据

- 4. Release manifest
- 5. 评测报告 + 趋势
- 6. Trace 抽样 + Bad case
- 让审计员看“工程证据”
- 来源:Week 11/12/14

RISK · 风险缓解

- 7. 风险识别与缓解
- 8. 应急 / 回滚机制
- 让审计员信“出事能控”
- 来源:Runbook + Canary
- 配合:Week 12/14

老司机说

你数数这 8 段的“来源”那一行——全是你前面几周已经产出的东西。合规白皮书的本质,就是把这些工程产物,换一个“监管看得懂”的视角重新排列一遍。所以我说:合规是工程治理的“商业化输出”,你治理做扎实了,白皮书是自然产物、一条命令 30 秒生成,而不是外包 \$20k+、耗时三周的苦工。

COMPLIANCE GENERATOR

合规白皮书自动生成器——从 manifest + 评测 + trace + 血缘 汇聚

一条命令, 同一个 release 可生成 EU / 中国 / NIST 三份不同视角

```
# release/compliance/generator.py
def generate_compliance_whitepaper(release_id, framework="eu_ai_act"):
    manifest = load_release_manifest(release_id)          # Week 14 L2
    # 1-3 段 · 系统概览
    overview = {"system_description": load_system_blueprint(),
                "data_sources": extract_data_contracts(manifest["spec"]["data"]),
                "models": extract_model_info(manifest["spec"]["model"])}
    # 4-6 段 · 工程证据
    evidence = {"release_manifest": manifest,
               "evaluation_report": {"baseline": manifest["spec"]["eval"],
                                    "trend_30d": fetch_metric_trends(release_id, 30)},
               "trace_samples": sample_traces(release_id, n=100, bad_cases=True)}
    # 7-8 段 · 风险 + 缓解
    risk = {"mitigations": {"rollback": "1-click", "canary": manifest["spec"]["rollout"],
                           "red_lines": ["PII=0", "block_rate>=99%", "citations"]},
           "incident_history_6mo": fetch_postmortems(months=6)}      # Week 12
    wp = {"release_id": release_id, "framework": framework,
          "overview": overview, "evidence": evidence, "risk": risk}
    wp["digest"] = sha256(json.dumps(wp, sort_keys=True).encode()).hexdigest()
    return wp # 不可篡改
# $ omni compliance generate --release rag-v2026.05.18-001 \
#   --framework eu_ai_act --notarize sigstore://...
```

老司机解读

overview/evidence/risk 三段 · 每段直接消费 Week 11-14 产物, 完全不靠人工填

trace_samples + bad case · 让审计员看"真实样本", 比人工写的"案例"可信得多

incident_history 来自 postmortem 库 · Week 12 学过, 自动汇聚半年事故和应对

framework 参数 · 同一 release 生成 EU + 中国 + NIST 三份不同视角白皮书

digest + Sigstore 公证 · 让白皮书不可篡改, 监管审计时就是"数字签名"

COMPLIANCE RED LINES

Copilot 的 4 类合规红线——触碰一次立刻冻结

白皮书是"事后给监管看", 红线告警是"事前不让它发生"

红线类别	具体指标	触发动作	法规依据
数据 · PII 泄露	PII 出现在输出中	立即冻结 + 电话通知合规	GDPR / 个保法
数据 · 越权访问	低权用户访问高权数据	立即冻结 + 安全审计	内控 + 行业监管
推理 · 违规话术	生成"保证赚钱"等话术	立即拦截 + 模型暂停	金融监管 / 反诈
行为 · 越权动作	Agent 调未授权工具	立即冻结 + Trace 保存	EU AI Act 透明度

踩坑提醒

这 4 类红线,绝不能靠人工盯——必须接 Week 12 告警 + Week 14 manifest 的 auto_rollback_on,24/7 自动响应。我见过一次 PII 泄露,因为没做自动冻结,从发生到人工发现过了 40 分钟,那 40 分钟里泄了几千条,事后光通知用户、报监管就折腾了一个月。红线这东西,一次就够你喝一壶,必须机器自动掐断。

RED LINE ENFORCEMENT

合规红线的工程强制——写进 manifest, 自动监控 + 链式审计

红线不是"建议", 是 manifest 里可执行的字段

```
# release/manifests/rag-v2026.05.18-001.yaml (升级 v6)
spec:
  compliance:
    framework: [eu_ai_act, china_备案_标识]
    risk_class: high          # EU AI Act 风险等级
    red_lines:
      - {id: RED-PII-001, metric: pii_leak_rate, threshold: 0,
        action: freeze_service + page_compliance}  # 零容忍
      - {id: RED-AUTH-001, metric: cross_tenant_access, threshold: 0,
        action: freeze_service + audit_log}
      - {id: RED-AGENT-001, metric: agent_unauthorized_tool_call, threshold: 0,
        action: freeze_agent + full_trace_archive}
    audit_chain:              # 链式审计,防篡改历史
    generator_digest: sha256:abc...
    signature: sigstore://ed25519/...
    previous_release: rag-v2026.05.17-003
    previous_digest: sha256:def...
    regulatory_reports:      # 备案号入 manifest
      - china_备案_id: CNAI-2026-1234
      - china_标识_compliant: true      # GB 45438 内容标识
```

老司机解读

red_lines 每条 = id+metric+threshold+action · 不是"建议",是"自动强制"

action 绑 Week 12 告警 + Week 14 rollback · 触发即执行,不等人

audit_chain · previous_release + previous_digest 形成链式审计,历史不可篡改

signature: sigstore · 把 digest 锚到签名服务,监管审计时是"数字公证"

标识_compliant · 2026 中国 C 端必填,GB 45438 内容标识合规入 manifest

THE 30-SECOND RESPONSE

监管来查的"30 秒响应" workflow——一键导出，加密签名

有工程治理：30 秒导出一份不可篡改白皮书；没有：找文档一周 + 手工拼



关键判断

一条 omni compliance export 命令, 给定"哪个版本 + 什么时间", 30 秒汇齐 manifest、评测趋势、trace 抽样(含 bad case)、半年 postmortem、白皮书, 全部 ed25519 签名 + RFC3161 时间戳 + Sigstore 锚定, 加密传输, 连"谁查的"都记进审计。按 EU AI Act 第 12 条, 这些留存 7 年。监管审查从"项目级紧急任务", 降级成"工程系统的日常输出"——这就是治理的最终价值。

INDUSTRY 2026

AI 合规自动化——2026 的工程红线

合规已从"软约束"变成"硬门槛": 工程治理不是先进做法, 是准入资格

来源	信号	给你的判断
EU AI Act	GPAI 执法 2026-08、透明度 2026-08 生效	进欧盟必须可追溯/可审计,没治理不能上
中国 GB 45438	AI 内容标识 2025-09 强制	C 端服务:备案 + 内容标识是硬门槛
Anthropic RSP / system card	每个 release 附风险评估 + 卡片	业界事实标准,发布必带
NIST GenAI Profile	把 AI 风险管理做成可操作框架	美国政府采购参考,自愿但加分

老司机说

一句总的判断:2026 年,AI 合规已经从"要不要做"变成"不做就上不了线"。但你别把它当成纯负担——换个角度,合规恰恰是把你前面所有工程治理"变现"的地方:同样一套 manifest + trace + 血缘 + 评测,对内是"排障和回滚的工具",对外就是"过监管、拿标、投标的资本"。治理做扎实的团队,合规是顺水推舟;治理欠账的团队,合规就是那道永远过不去的坎。

LESSON 05 · CANARY + AUTO ROLLBACK

灰度发布 + 自动回滚：让"AI 系统上线"从"祈祷"升级为"工程"

前 4 节建好了"版本 + 影响 + 合规"——但客户面对的不是 *manifest*，是 *production* 流量

真实场景 · "季度大版本"就是一场对着屏幕祈祷的赌博

我见过太多团队搞"季度大版本"——攒三个月的改动, *prompt*、数据、模型一次性全换, 选个周五晚上全量上线, 然后全组守在屏幕前祈祷别出事。这不叫工程, 这叫赌博, 而且是压上全部身家的那种赌。真正的做法反过来: 每一次变更——哪怕只改一个 *prompt* 字符——都走完整发布流程, 但流程要快、要自动: *Canary* 5% → 25% → 50% → 100%, 每一档机器自动看指标决定升还是回, 全程半小时、零人工守候。这一讲我教你, 怎么让 *Copilot* 上线变成"高频 + 低风险"的日常动作。

咱们这节聊 4 件事:

为什么"高频低风险"比"低频高风险"是 10 倍的工程进步

3 种发布模式怎么选, 4 阶段 *Canary* 每阶段看什么指标

Canary 自动决策框架 + 秒级回滚引擎——半夜 3 点也能自己 *promote/rollback*

2026 工具: *Argo Rollouts* / *Flagger* / *OpenFeature* + 在线 LLM 评测门禁

RELEASE OFTEN, RELEASE SMALL

AI 系统的发布频率应该高——但每次都低风险

SRE 文化的精髓：越频繁越安全，因为每次改动小、影响可控、回滚容易

传统 SRE 有句老话:release often, release small——越频繁反而越安全,因为每次改动小,出事也是小事,一眼能看出是哪次改的。AI 团队却常常反着来:憋大版本、一次改一堆,出事就是大事,还根本分不清是 prompt 的锅、数据的锅还是模型的锅。正确做法:每次变更都走完整发布流程,但流程要快——Canary 自动决策,5→25→50→100%,30 分钟全自动。

Big Bang(反)

- 一次性 100% 替换
- "季度大版本"
- 风险:极高
- 回滚代价:极高
- 反模式——避免

Blue-Green

- 新版本部署完 → 100% 切
- 可秒级回滚
- 风险:中(无渐进)
- 成本:2x 基础设施
- 适合:基础设施类

Canary(推荐)

- 5→25→50→100% 渐进
- 自动决策升档
- 风险:最低
- 成本:与流量成正比
- AI 系统首选 ★

FOUR CANARY STAGES

AI 系统的 4 阶段 Canary——每阶段决策标准不同

一刀切的"Canary 跑 1 小时"反而风险更大，必须分阶段精细化

阶段	流量	观察时长	关键指标	升档条件
Stage 1	5%	15-30 min	红线(P0/合规/严重错)	无 P0 + 无合规违反
Stage 2	25%	1-2 h	+ 质量(faithfulness)	指标不显著退化
Stage 3	50%	2-4 h	+ 业务指标 + 成本	SLO 全部 pass
Stage 4	100%	N/A	持续监控	全量发布完成

关键判断

注意每阶段"关键指标"是层层加码的:5% 时只看红线(快速止血),越往上看看得越全(质量、业务、成本)。为什么不一上来全看?因为 5% 流量样本太小,质量指标噪声大,你拿它做决策就是误判。先看能一票否决的硬红线,流量上去了、样本够了,再看软指标——这个顺序,是 Canary 精细化的精髓。

CANARY DECISION FRAMEWORK

Canary 自动决策框架——不靠人工守，半夜 3 点也能跑

每 5 分钟跑一次，输出 *promote / hold / rollback*

```
# rollout/canary.py
def evaluate_canary(release_id, stage, current_traffic_pct) -> CanaryDecision:
    canary = collect_metrics(release_id, stage, window_minutes=15)
    baseline = collect_metrics("prev_stable", stage=4, window_minutes=15)

    # 1. 红线优先——任何 stage 触碰立即回滚,不等观察期
    if violations := check_red_lines(canary):
        return CanaryDecision("rollback", f"Red line: {violations}", canary)

    # 2. 阶段特定检查——与 baseline "对比退化",不是绝对阈值
    for metric, threshold in STAGE_CONFIG[stage]["promote_if"].items():
        if metric.endswith("_rate"):
            # 越低越好
            if canary[metric] > baseline[metric] * (1 + threshold):
                return CanaryDecision("rollback", f"{metric} regressed", canary)
        else:
            # 质量类,越高越好
            if canary[metric] < baseline[metric] * (1 - threshold):
                return CanaryDecision("rollback", f"{metric} dropped", canary)

    # 3. 最少观察期没到 → 保持
    if time_since_stage_start(release_id, stage) < STAGE_CONFIG[stage]["min_observe"]:
        return CanaryDecision("hold", "min observe not reached", canary)

    return CanaryDecision("promote", "all checks passed", canary) # 升档
```

老司机解读

red_line 优先 · 任何阶段触碰立即回滚,不让红线"等观察期"

baseline 对比 · 不用绝对阈值,而是跟上一版"对比退化",避免季节性/流量波动误判

min_observe · 每阶段强制最少观察时长,防止"5 分钟一切正常"就草率升档

STAGE_CONFIG 按阶段配 · Stage1 只查红线、Stage4 查所有 SLO,精细化

每 5 分钟跑一次 · Canary 全程不用人介入,半夜 3 点也能正常 promote/rollback

ROLLBACK MECHANISMS

3 种回滚机制——速度差 100 倍

前 13 周做的所有原子绑定，就是为了让回滚从"小时级"降到"秒级"

机制	具体做法	速度	依赖
重部署回滚	回退代码 + 重启 + 重建索引	小时级	基础设施
配置切换	改配置 + 滚动更新	分钟级	配置中心
release_id 切换	切 router 指向旧 release_id	< 60 秒	Week 14 整套基础

关键判断

看清这个 100 倍的差距怎么来的:重部署要重建一大堆东西,慢;配置切换快一点;而 release_id 切换,之所以能做到 60 秒内,是因为前 13 周你把 6 类对象都原子绑进了一个 release_id——回滚时只需要把 router 的指针,从新 release 切回旧 release,一个指针的事。

所以我常跟团队说:Week 14 前面那些绑定、血缘、manifest,看着琐碎,但它们全部是为了这一刻——出事时,你能在客户还没骂完之前,一键切回去。一个中等体量的 C 端服务,P0 故障每分钟损失常在 \$500 到 \$5,000,4.7 秒和 3 小时的差,就是几十万的差。

ROLLBACK ENGINE

秒级回滚引擎——6 类对象在一个事务里原子切换

Week 14 整周工程的最终输出：一份 release_id 锁定 = 一次原子切换

```
# rollout/rollback.py
def emergency_rollback(current, target, reason, triggered_by) -> dict:
    t_start = time.time()
    tgt = load_release_manifest(target)
    assert tgt["status"] in ["archived", "stable"]    # 目标必须健康

    # 关键:6 类对象必须在一个事务里切换,部分成功 = 大事故
    with traffic_router.transaction():
        traffic_router.set_release(target)
        # data: lakeFS 分支切到 target.data.lakefs_tag
        # index: pgvector 指向 target.index.snapshot
        # prompt/model/skill_pack/graph: 全部切到 target
        cache.invalidate(["service_config", "release_pointer"])

    time.sleep(10)    # 10s 健康观察窗
    assert check_health(target)["error_rate"] < 0.01

    audit_log.write({"event": "release.rollback", "from": current, "to": target,
                    "reason": reason, "duration_seconds": time.time()-t_start})
    notify_oncall(f"ROLLBACK: {current} → {target}")
    return {"success": True, "duration_seconds": time.time()-t_start}

# $ omni release rollback rag-v2026.05.17-003 --triggered-by auto_canary
# ✓ Rollback completed in 4.7s
```

老司机解读

traffic_router.transaction() · 6 类对象切换必须在事务里, 部分成功 = 大事故

一份 release_id = 一次原子切换 · 这是 Week 14 整周工程最关键的回报

check_health 10s · 切换后立刻验证,防止"切到一个本身就问题的旧版本"

audit_log 记全程 · 谁触发、为什么、耗时多少,事后可复盘

4.7 秒 vs 没治理团队的 30 分钟到 3 小时 · 这就是"工程治理 ROI"最直观的数字

DEPLOYMENT MATURITY

AI 系统发布成熟度的 5 级——你的团队在哪一档

大部分团队卡在 L1-L2，升到 L4 不用重做基础设施，把本周工具组装起来就行

等级	具体表现	回滚耗时	事故频率
L0 · 手动	手敲命令 / 改配置	小时级	月 1-3 次
L1 · CI/CD	代码自动部署,数据/Prompt 手动	30 分钟	月 1-2 次
L2 · 蓝绿	完整版本切换,无渐进	分钟级	季 1-2 次
L3 · Canary 半自动	分阶段但人工守	< 10 分钟	半年 1 次
L4 · Canary 全自动	5-100% 全自动决策	< 1 分钟	年 1 次

老司机说

你对着这张表给自己团队打个分——大部分我见过的团队,老实说都在 L1 到 L2 之间:代码能自动部署了,但数据、Prompt 还是手动改、手动回滚。从 L2 升到 L4,不需要你重做基础设施,需要的只是把 Week 14 这五节的工具(lakeFS + manifest + 影响分析 + Canary + 回滚引擎)组装起来。工具都是现成的,差的是"把它们串成一条自动链"的决心。

END-TO-END DEPLOY

完整发布 workflow——Week 14 收口，一个 push button

把 Week 1-14 所有工程能力收口成“按一下 release，剩下全自动”



关键判断

这条 workflow，就是这门课前 14 周所有功夫收口的地方：工程师在 PR 里点一下 merge，后面全自动——lakeFS 合分支、生成 manifest、跑影响分析、多人审批、4 阶段 Canary 自动决策、100% 后自动生成合规白皮书。从“点 release”到“100% 流量稳定”，全程 60-120 分钟、零人工守候。这才是“AI 系统能上生产”真正的样子。

INDUSTRY 2026

AI 系统 Canary 发布的行业实践——工具早已开箱即用

2026 关键不是工具，是建立"频繁低风险发布"的工程文化

来源	做法	给你的启示
Google SRE	Canary + 自动决策 + 错误预算	本周所有理论的基础,直接照搬
Argo Rollouts / Flagger	K8s 上把渐进发布做成 GitOps	改个 yaml,系统自动 Canary,别手写
OpenFeature (CNCF) / LaunchDarkly	特征开关标准 + 灰度 SaaS	不想自建灰度的团队,直接用
在线 LLM 评测门禁	Braintrust/LangSmith 把评测卡进发布	2026 新增:Canary 阶段跑在线评测当门禁

老司机说

2026 有个 AI 专属的新东西值得你注意:把"在线评测"直接卡进 Canary 门禁。传统 Canary 看的是延迟、错误率这些系统指标,但 AI 系统真正的风险是"答得对不对",这是个软指标。所以现在成熟团队在 Stage 2/3 会挂一个在线的 faithfulness 抽样评测,质量掉了自动回滚——这是 Week 11 评测和 Week 14 发布合体的最新形态。工具都现成了,难的从来不是工具,是你敢不敢把"季度大版本"的旧习惯改掉,真正做到"一天发十次、每次都不慌"。

WEEK 14 · WRAP UP

Week 14 五节课的总收口——治理体系完整成型

让 AI 系统具备“软件工程级”的版本治理能力，这是从实验工具到商业产品的最后一公里



关键判断

这五节连起来,就是一套完整的 AI 系统治理方法论:数据能 Git(Branch)、版本能原子绑定(Bind)、改动前能看影响(Impact)、上线能过监管(Compliance)、发布能灰度秒回滚(Canary)。走完这一周,你的 Copilot 第一次真正具备了“持续演进 + 监管可审 + 事故可控”的工程级治理能力——这就是“实验室 demo”和“能上生产、能卖钱的产品”之间,那道最难跨的坎。

WEEK 14 · 收口

Week 14 的 5 个判断——AI 治理达到软件工程级

下周 Week 15: 成本性能与上线收官, 课程的最终交付

- Branch:数据 Git 化——lakeFS 把 PR/Merge/Revert 搬到数据,2026 目录用 Polaris/Unity
- Bind:6+ 类对象原子绑定——release manifest 是主键,2026 加签名溯源
- Impact:血缘 + 影响分析——改之前先看下游,列级血缘 + 接 PR,告别开会评估
- Compliance:合规白皮书 = 工程产物自动汇聚——2026 别讲错(EO14110已废/中国AI标识强制)
- Canary:4 阶段灰度 + 自动决策——秒级回滚是前 13 周原子绑定的最终回报

下一讲

下一讲 · Week 15 Capstone:前 14 周把完整的企业级 AI 数据工程体系建好了。最后一周收官——成本性能优化、SLO 体系、运维 Runbook、完整作品集交付,让 Copilot 真正能长期稳定跑下去。

WEEK 14 · THE END

你的 AI 系统，已经达到软件工程级

从数据分支化到原子绑定，从血缘到合规白皮书，再到秒级回滚——AI 系统第一次具备了“持续演进 + 监管可审 + 事故可控”的工程级治理能力。这是从“能跑”到“能交付”的最后一公里。

本周交付物（已 push 至 GitHub 仓库）

lakeFS Setup

data/lakefs/config.yaml

Release Generator

release/generator.py

Impact Analysis

tools/impact_analysis.py

Compliance Gen

release/compliance/generator.py

Canary Framework

rollout/canary.py

Rollback Engine

rollout/rollback.py

下周 → Week 15 成本性能与上线收官：课程的最终交付,让 Copilot 长期稳定跑下去。我们下周见。